

脆弱性情報の上流は安定していない： 公開前検査による 116 日間の全数観測

篠原 俊一^{1,a)} 中岡 典弘^{1,b)} 神戸 康多^{1,c)}

概要：脆弱性データベースは多数の上流データソースを集約して構築されるが、上流は必ずしも安定せず、データの消失や公開済みアドバイザリの遡及的な書き換えが現実に行われている。本研究では、この不安定性を定量観測するため、OSS 脆弱性スキャナ Vuls の DB 公開 CI に公開前検査を実装した。検査は公開候補と直前の公開版を、検知結果と検知条件構造の 2 観点で比較し、変動率が閾値を超えた場合は公開を保留して人間の確認に委ねる。116 日間の本番運用の全数調査では 69 件のイベントで検査が発動し、全工程を履歴管理する既報の基盤を用いて、その全てについて原因を上流データの変化か、変換や DB 構築や取得設定といった自前の処理かのいずれかに帰属できた。日常変動と構造的イベントは変動率で桁が分離しており、閾値による単純な判定が実用になること、また欠損 DB の公開を 2 回、異常な候補 DB の公開を 1 回阻止したことを示す。

キーワード：脆弱性データベース、データ品質、公開前検査、履歴管理、ソフトウェアサプライチェーン

Vulnerability Feeds Are Not Stable: A 116-Day Exhaustive Observation via Pre-Publication Inspection

SHUNICHI SHINOHARA^{1,a)} NORIHIRO NAKAOKA^{1,b)} KOTA KANBE^{1,c)}

Abstract: Vulnerability databases are built by aggregating many upstream data sources, but these upstreams are not always stable: data disappears, and published advisories are retroactively rewritten. To quantify this instability, we implemented a pre-publication inspection in the CI pipeline that builds and publishes the database of Vuls, an open-source vulnerability scanner. The inspection compares each publication candidate with the previously published version in terms of detection results and detection-criteria structure, and when the change rate exceeds a threshold, it withholds publication pending human review. In an exhaustive study covering 116 days of production operation, the inspection was triggered by 69 events, and our previously reported infrastructure, which keeps every pipeline stage under version control, allowed us to attribute every event to either upstream data changes or our own processing such as data transformation, database construction and fetch configuration. We show that everyday fluctuation and structural events are separated by an order of magnitude in change rate, making this simple threshold-based inspection practical, and that the inspection blocked the publication of databases with missing data twice and of an anomalous candidate once.

Keywords: Vulnerability Database, Data Quality, Pre-Publication Inspection, History Management, Software Supply Chain

1. はじめに

脆弱性対応の実務は、ディストリビュータやベンダが公開するアドバイザリや脆弱性フィードを「正」として組み

¹ フューチャー株式会社

Future Corporation

a) s.shinohara.yx@future.co.jp

b) n.nakaoka.46@future.co.jp

c) k.kanbe.r4@future.co.jp

立てられている。脆弱性スキャナはこれらを集約した脆弱性データベース（以下、脆弱性 DB）に依存し、検知結果はフィードの内容をほぼそのまま反映する。ここには、上流フィードは概ね安定しており、変化は新規脆弱性の追加という単調な形をとる、という暗黙の仮定が置かれている。この仮定はどの程度正しいのだろうか。

我々は OSS 脆弱性スキャナ Vuls[1] のエコシステムにおいて、40 超のデータソースから脆弱性 DB(vuls.db) を CI で自動ビルドし、6 時間おきに公開している。前稿 [2] では、この収集 (fetch)、変換 (extract)、構築 (db build) の全工程をバージョン管理システムと連携させ、脆弱性情報の変更履歴を客観的事実として追跡・再現可能にする履歴管理基盤を報告した。本稿はその続編にあたる。前稿の基盤が「脆弱性情報の変化を記録できる」ことを示したのに対し、本稿はその記録能力を使って「上流は実際にどう変化しているか」を系統的に測定し、さらにその測定を DB の公開品質管理として実用化した結果を報告する。

測定の装置は単純である。公開直前の候補 DB を、現に公開されている直前版 (baseline) と自動比較し、変動率が閾値を超えたら公開を保留して人間に見せる。我々はこれを公開前検査 (diff guard) と呼ぶ。導入の直接の動機は、自らのパイプライン変更と上流の破損データの複合、および DB とスキャナのバージョン非互換によって「壊れた DB」を公開してしまった 2 件のインシデント (2.2 節) であり、検査は第一義には安全ネットである。しかし 116 日間の本番運用で得られた発動記録の全数調査は、副産物として、脆弱性情報の上流がいかに大きく、頻繁に、時に黙って動くかの縦断観測データとなった。

本稿の貢献は次の 3 点である。

- (1) **上流不安定性の実測** (5 節, 6 節): 116 日間・943 run の全数調査から 69 件の発動 episode を抽出し、59 件を上流由来、10 件を自前の処理由来に帰属した上で、上流の不安定性を分類・定量化した。観測されたイベントは、可用性の不安定 (月次データの消失とフラッピング)、完全性の危うさ (遡及的な再キュレーションやアナウンスのないサーバ側再編)、スケールの分布 (日常変動と構造的イベントの析分離) の 3 群に整理できる。これらの上流の多くは他の脆弱性ツール (Trivy, Grype, OSV 系等) も共有しており、知見はエコシステム横断で再利用可能である。
- (2) **原因帰属の方法論** (4 節): 発動の原因をビルダ、抽出器、取得設定、上流の順に容疑を除外して確定する切り分け手順を定義し、全イベントに適用した。各ステップは履歴管理基盤 [2] の対応する工程の履歴 (DB メタデータ、抽出器のコミット履歴、raw データの Git 差分) に依拠しており、全工程の履歴管理が帰属の成立条件であることを示す。
- (3) **公開前検査の設計と運用機構** (3 節): 検知結果差分と

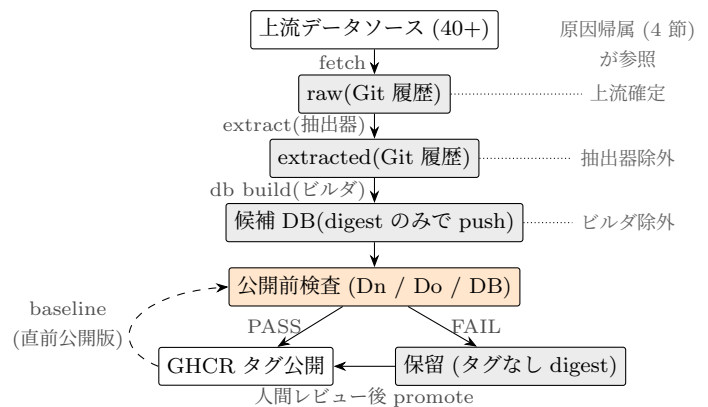


図 1 脆弱性 DB 公開パイプラインと公開前検査。全工程が Git 履歴管理 [2] され、原因帰属 (4 節) は各工程の履歴を参照する。

DB 構造差分の 2 観点による検査本体に加え、監査証跡付き手動 override(promote)、対象別閾値の統計的導出と定期的な再導出 (grooming) という、検査を運用し続けるための機構を設計・実装した。検査は欠損または異常な候補 DB の公開を 3 回阻止した。

2. 背景

2.1 履歴管理基盤 (前稿) の要点

vuls.db の生成は 3 工程からなる [2], [3], [4]。fetch 工程は一次情報源から取得したデータを原形をほぼ保った vuls-data-raw 形式で、extract 工程はそれを共通スキーマの vuls-data-extracted 形式で、それぞれ Git 管理下に置く。db build 工程は extracted データから BoltDB 形式の vuls.db を構築し、コンテナレジストリ (GitHub Container Registry, 以下 GHCR) にタグおよび digest で公開する。以下本稿では、extract 工程の変換コードを抽出器、db build 工程の構築コードをビルダと呼ぶ。この構成により、(1) 任意時点の上流データの状態をコミット単位で遡れる、(2) DB を digest で固定して検知結果を再現できる、という 2 つの性質が得られている。本稿の測定はこの 2 性質の直接の応用である。

2.2 動機となった 2 つのインシデント

2026 年 3 月、生成物としての DB が壊れているのに公開が成功してしまう事象を 2 件経験した。

インシデント 1(RHEL VEX, false negative): 我々がパイプラインにデータ形式変更を投入したのと同じ日に、上流から不正な VEX(Vulnerability Exploitability eXchange) データが配信された。変換はエラー終了してデータが旧形式のまま残り、新形式を前提とする後段フィルタが空データを生成した。結果、redhat:9 で本来検知されるべき CVE の 51.9%(3,387 件) が「影響なし」となる DB が公開された。どちらか一方だけなら実害はなかった: 不正データがなければ変換は新形式へ移行して整合し、形式変

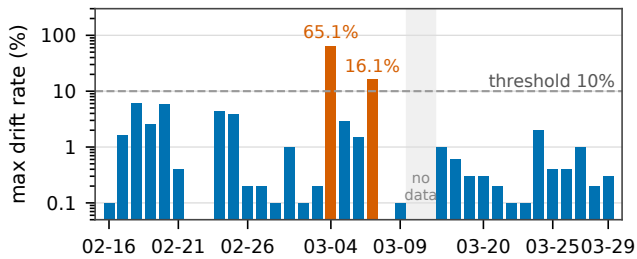


図 2 導入前ベンチマーク: 連続 1 日ペア 35 組の最大変動率 (対数軸, 破線は db 既定閾値 10%). 橙は実在の上流イベント (Red Hat VEX 65.1%, SUSE OVAL 16.1%) で, 正常帯 (中央値 0.3%) と桁で分離する。

更がなければ変換失敗はデータが一時的に古いままになるだけで済んだ。工程間の形式整合を欠いた我々の実装と上流の異常が複合して初めて成立した欠陥である。

インシデント 2 (Ubuntu vulnerable: false, false positive): DB 側に導入されたマーカーを, それを解釈するフィルタを持たない旧バージョンのスカナが読んだ結果, ubuntu.2204 の検知件数が 5,968 件から 12,296 件 (+106.0%) に倍増した。DB 単体でもスカナ単体でも正常であり, 組み合わせの非互換が原因である。

従来の CI が検査するのはビルドの成否であって, 生成物のデータとしての妥当性ではない。また 2 件は対照的で, 前者は DB 単体を見れば検出できるが, 後者は DB とスカナ (の特定バージョン) の組で実行しなければ検出できない。この 2 面性が次章の設計に直結する。

3. 公開前検査の設計

3.1 導入前ベンチマーク: 閾値検査の成立可能性

脆弱性 DB は日々更新されるため差分ゼロはあり得ず, 閾値による検査が成立するには正常な変動と異常なイベントが変動率で分離する必要がある。導入前に, GHCR 上に履歴として残っている過去 37 日分の DB (2026-02-15 から 03-29) を用い, 連続 1 日ペア 35 組 (DB が残存しない 6 日分は欠測) で DB 構造差分を実測した。通常運用の変動率は中央値 0.3%, 最大 6.2% に収まり, 期間中に 10% を超えた 2 組はいずれも実在の上流データ品質イベント (Red Hat VEX 構造変更で redhat:6 が 65.1%, SUSE OVAL 大規模更新で 12~16%) であった。正常帯と異常イベントが桁で分離するというこの観測が, 閾値検査成立の根拠である。なお過去任意時点の DB ペアでベンチマークできること自体, 公開物が digest 付きで履歴化されている [2] ことの恩恵である。

3.2 3 つのチェック

公開 CI の圧縮後・公開前に, 次の 3 チェックを実行する (検知結果差分の 2 つを Dn (新スカナ) と Do (旧スカナ), DB 構造差分を DB と略記する)。

- **Dn: 検知結果差分 (スカナ最新版).** baseline と target の両 DB に対して同一のスカナ結果 fixture 群 (実機スカナ由来 75 ファイル, Windows 系を含む 12 OS ファミリー + CPE 系 6 ソース) でスカナを実際に実行し, ファイル単位で検知 CVE 集合を比較する。fixture 外の対象の検知退行は DB チェックのみが検出する。変動率は (追加数 + 削除数) / baseline 検知数で定義する。
- **Do: 検知結果差分 (固定した旧バージョンのスカナ).** 過去バージョンとの組でのみ現れる非互換退行 (インシデント 2 型) を捕捉する。
- **DB: DB 構造差分.** BoltDB を直接開き, ecosystem (後にソース単位へ細分化) ごとに検知条件木を leaf Criterion レベルまで平坦化して構造比較する。スカナ結果もスカナも不要で DB のみで完結する。比較粒度をバイト列ではなく Criterion にしたのは, JSON キー順などの無害な変更での偽陽性を避けつつ, 分母 (全体で約 330 万 Criterion) を大きく取り正常変動を小さな drift として吸収するためである。

各チェックは対象 (fixture ファイルまたは DB 上の対象) ごとの変動率を 1 行として出力し, 1 行でも閾値超過なら run が FAIL する (baseline 不在の対象は 100% と扱う)。チェック結果は変動率, 増減の内訳, 変動した CVE やアドバイザリの ID を含む構造化されたレポートとして常に CI の Job Summary に出力される。これは前稿が課題として挙げた「行ベース差分ではなく論理的な構造としての差分提示」[2] の, 公開品質管理という文脈での実装形でもある。導入当初は最初の FAIL で検査が停止する実装だったが, 先に FAIL したチェックが後段のシグナルを隠す問題が判明し, 現在は 3 チェックすべてを実行してから集約判定する。

3.3 FAIL 時の運用: 監査証跡付き手動 promote

候補 DB は検査実行前に digest のみ (タグなし) で GHCR に push しておき, PASS 時にのみタグを付けて「公開」する。FAIL した場合, 候補はタグなしのまま digest で取得可能な状態で残るため, 事後検証が可能である。オペレータがレポートを確認し正当な変更と判断した場合は, 専用 workflow で該当 digest に手動でタグ付けする (promote)。実行者, digest, タグが run 履歴に残り, 「誰がどの候補を override したか」が追跡できる (判断対象のレポートは FAIL run 側に残る)。運用ルールとして, promote の判断と実行は人間に限定している (AI エージェントには実行させない)。

3.4 閾値の per-target / per-source 化と grooming

閾値はデフォルトで detection 5% / db 10% とし, 運用実績に基づき対象別に緩和する override 機構を持つ (db

10%は導入前ベンチマークに基づき、detection 5%は経験的な初期値である)。導入1ヶ月の運用で、新ディストリビューション立ち上がり期の一括 triage, rolling release, ベンダ月次サイクルといった反復的な正当変動が毎回検査に引っかかることが判明したためである。override 値は失敗履歴の統計から「観測ピーク+ヘッドルーム」で導出し、単発スパイクは override せず FAIL させて人間に見せる(例: EOL ディストリビューションの39%変動は異常として残置)。リストの陳腐化を防ぐため約2ヶ月周期で全再導出する(grooming)。さらに運用後期、ecosystem 一括の変動率では大きなソースがノイズフロアになって小さなソースの異常を希釈する問題が顕在化し(6.3節)、閾値もレポートもソース単位に細分化した。なお変動率は追加と削除の対称量であり、override の緩和は検知漏れに直結する削除側の感度も同時に下げる。両方向を分けた非対称閾値は今後の課題である。

4. 測定方法論: 全数調査と原因帰属

全数調査: 本番導入(2026-04-23)から08-16までの116日間、2系統(安定版 / 開発版)の全943 runを対象に、run ログ、PR、promote 履歴をGitHub APIで全数収集した。FAIL 中は promote があるまで baseline が動かず、同じ差分で6時間おきの cron が再 FAIL し続けるため、run 数は重複を含む。そこで連続 FAIL を1つに潰した fail sequence(104件)、さらに2系統の統合とソース単位への分解を同時に行った source-episode(69件)をイベントの正準単位として定義した。episode は24時間以上の途切れで分割する集約規則であり、上流イベントの独立性の保証ではない(分割間隔を12h / 48h にすると77 / 62件になる)。episode のソース族は全期間 target 名から推定し、後半窓ではレポートの明示ソースで内訳を検証した。

原因帰属: 各イベントについて、次の順で容疑を除外して原因を確定する。

- (1) **ビルダ除外:** baseline と target の両 DB のメタデータに記録されたビルダのバージョンを比較する。一致すれば DB 構築コードは無罪。
- (2) **抽出器除外:** 対象期間の抽出器リポジトリのコミット履歴を走査する。当該ソースの fetch / extract コードに変更がなければ変換コードは無罪。
- (3) **取得設定除外:** 同期間の vuls-data-db(取得設定を持つリポジトリ)のコミットを走査する。取得対象の seed リストや新規データソースの追加がなければ取得設定は無罪。
- (4) **上流確定:** raw データの Git 履歴から対象期間のコミットを特定し、差分の中に変動と件数・ID が一致する変更実体(smoking gun。例えば新規アドバイザリファイル群やディレクトリの消失)を突き止める。

この手順の各ステップは、履歴管理基盤[2]の対応する

工程の履歴にそれぞれ依拠している。ビルダ除外は DB メタデータ、抽出器除外はコードのコミット履歴、上流確定は raw および extracted データの Git 履歴である。全工程が履歴化されていなければ、例えば「上流の一時障害なので候補を破棄すべき」と「正当な変更なので promote してよい」の区別(5.2節)は当て推量になる。逆に言えば、履歴管理は本測定の成立条件であり、前稿の基盤の実証でもある。実際、全69 episode をいずれかの工程に帰属し、「原因不明」は残らなかった。ただし証拠の水準は様ではない。自前の処理由来と判定した10件のうち6件は原因のコミットや PR の差分を直接の証拠として特定したが、残る4件は開発版で先行した CPE 関連変更系列への帰属であり、原因の工程までは確定していない。上流由来と判定した episode は、ビルダ、抽出器、取得設定をコミット履歴の走査で除外した上での帰属であり、raw 差分の smoking gun まで確認したのは代表事例に限られる。なお観測装置である検査自身も期間中に進化しており(3チェックの全実行化、fixture と override の追加、ソース単位化)、本データは固定した測定器による同質標本ではなく、進化する本番検査の発動履歴である(発動件数の期間内比較はこの影響を受ける)。また帰属手順は再現可能な triage 手順書として整備し、AI エージェント用のスキルとしても記述している(役割分担は6.6節)。

5. 運用結果

5.1 規模と原因分布

943 run 中、失敗は476 run。うち420 run が検査のステップで失敗した(閾値超過418+検査内インフラ障害2。残る56はビルド工程39とrunner障害等17)。イベント単位では69 source-episode である。チェック別の捕捉は DB 単独23件、Dn 単独16件、複数同時30件、**Do 単独0件**である(6.4節)。原因の分布を表1に示す。内訳は上流由来59件、自前の処理由来10件である。自前由来のうち6件は原因のコミットや PR まで個別に特定し、残る4件は開発版で先行した CPE 関連変更系列への帰属で工程は未確定である。

観測期間の後半では、非上流由来のイベントが4件発生した。2件は前半に存在しなかった**取得設定由来**である。我々自身が取得対象の seed リストを拡張した直後(08-05~06のWindows更新プログラム(KB) seed 登録)に microsoft/microsoft-msuc が211.3%で発動し、新規データソースを追加した直後(08-10)には microsoft/microsoft-servicing が baseline 不在のため100.0%で発動した。上流にも抽出器にも変更はなく、「何を取りに行くか」の設定変更が原因であり、帰属手順に vuls-data-db 側のコミット走査(4節の手順3)を加えることで初めて分離できた。残る2件は**抽出器由来の意図的な変更**の再来である。代表例では、2012年から2022年の履歴的アドバイザリに検知

表 1 episode の代表判定の分布 (計 69 件). 複数原因を含む episode は公開可否を左右した原因で代表させた. 系列推定の 4 件は開発版で先行した CPE 関連変更群への帰属で, 工程 (抽出器かビルダか) は未確定. 他の自前由来 6 件は原因のコミットまで特定済み. 上流由来 (a) は fedora:46 のような小規模 baseline 起因の構造的 FAIL を含む. 捕捉列は発動したチェック.

判定	件数	代表例	捕捉
上流由来 (a) 正当な変更	55	月次パッチ等	混在
上流由来 (b) 一時的障害	2	CVRF 消失	Dn
上流由来 (c) 恒久的品質イベント	2	errata 再編	全て
抽出器由来 (コードバグ)	1	非決定的出力	DB
抽出器由来 (意図の変更)	3	検知の補完	DB
自前由来 (系列, 工程未確定)	4	開発版 cpe 系列	DB
取得設定由来	2	seed 追加	DB
ビルダ由来	0	(事例無し)	–

を補完する抽出器変更 (検知を持つファイルが 643 件から 1,065 件に増加) のマージ直後に, cpe.fortinet/fortinet-cvrf が 135.5%で発動した. こちらは既存の抽出器コミット走査 (同節の手順 2) で特定できる.

5.2 公開を阻止した 3 回

(1) **Microsoft CVRF 月次データ全消失 (2 回)**(CVRF: Common Vulnerability Reporting Framework): 上流 API が当月分のセキュリティ更新ドキュメント (700~1,300 ファイル) を丸ごと返さなくなり, Windows 系 13 ターゲットで最大 53.6%の削除のみの差分が発生した (2 回目は当月 CVE 約 1,270 件分の検知消失に相当). 公開されていればインシデント 1 と同型の大量 false negative である. 検査は候補を破棄させ, 上流回復後の再実行のみで復旧した. raw 履歴上, 当月ファイル数は 1273 → 0 → 1273 → 0 → 1278 とフラッピングしており, 単発の取得失敗ではなく上流配信自体の不安定であった (フラップの認定自体が履歴管理の産物である).

(2) **抽出器の非決定性バグ**: raw の変更は 266 ファイルなのに extracted は 23,163 ファイル動くという異常を DB チェックが 274.5%の drift として検出した. 原因は抽出器が型の取り違えでソート処理をスキップし出力順序が非決定になっていたことで, 修正 PR に帰結した. 期間中唯一の「上流のせいではないコード起因の異常」の捕捉である.

なお発動しないまま公開された破損は, 利用者報告と事後の再分析の範囲では確認されていない (網羅的な検証手段は無い).

5.3 防ぎ切れなかった事例と検査改良の契機

発動したが被害を防ぎ切れず, 検査と運用の改良に帰結した事例が 2 件ある.

(1) **AlmaLinux errata の縮退**: errata フィードが 1 週間に複数回再編成され, アドバイザリ数が 279 から一時 60 まで減少, 削除されたアドバイザリが参照していた 434 CVE のうち 87%(376 件) はフィード全体から消失し, 残存アドバイザリでも kernel 更新の検知条件が 73 から 2 パッケージへ縮退した (変動率は縮退時 99.2%, 後述の巻き戻し時に最大 428.3%). 過去バージョン系列に対する大量の false negative に直結する. 復旧作業で FAIL の滞留を解消する際にオペレータが縮退候補を promote してしまい, 縮退 DB は数時間公開に達したが, 取り込みを縮退前のコミットにピン留めして同日中に巻き戻し, 以後の流入を停止した. その後, 上流の HTML index に残るのにフィードから消えたアドバイザリだけを取得履歴から fetch 時に復元する機構を組み込みアンピンした. ピン留めも復元も, 取り込みの履歴管理が可能にする操作である (原因は 6.2 節).

(2) **VulnCheck 一斉ロールアウト**: 単一コミットで 145,672 ファイル, +957 万行という生成 CPE(Common Platform Enumeration) 設定の一斉付与を DB チェックが検出した. ただし当時の ecosystem 一括の変動率では「cpe 28.9%」としか言えず, 候補は promote された. 事後のソース単位の再分析で, 当該ソース単体では 189.6%の変動であり, かつ実検知が 60~75%消失する退行を含んでいたことが判明した. この教訓がソース単位への細分化 (3.4 節) の直接の動機である. 「発動したが分解能不足で判定を誤りかけた」事例として記録する.

2 件とも検査は発動しており, 止め切れなかったのは人間の承認判断である. 残存リスクは検出側より承認側にあり, 削除方向の自動ブロックや大規模消失時の複数名承認が次の改善点である.

5.4 false positive 論と運用ループ

発動の大半は正当な上流変更であり, これを「誤報」と見るかは設計思想による. 本検査は安全ネットであり, 822 CVE を単一アドバイザリに同梱するカーネル更新や, 約 600 パッケージを束ねた単一アドバイザリ (5,312 検知条件) のマス更新は, 正当であっても公開前に人間が一目見るに値する. 一方, 反復性が確立した既知の変動は override で恒久緩和する. 116 日間の手動 promote は 92 unique digest に達し, この規模が 6.5 節の運用コストの実体である.

6. 考察: 観測された上流の危うさ

6.1 消える・戻る・また消える (可用性)

月次データの丸ごと消失 (5.2 節) は 2 ヶ月連続で発生し, 2 回目は 2 日間に消失と復活を 4 回繰り返した. フィードの取得は「ある時点のスナップショット」であり, 欠損の

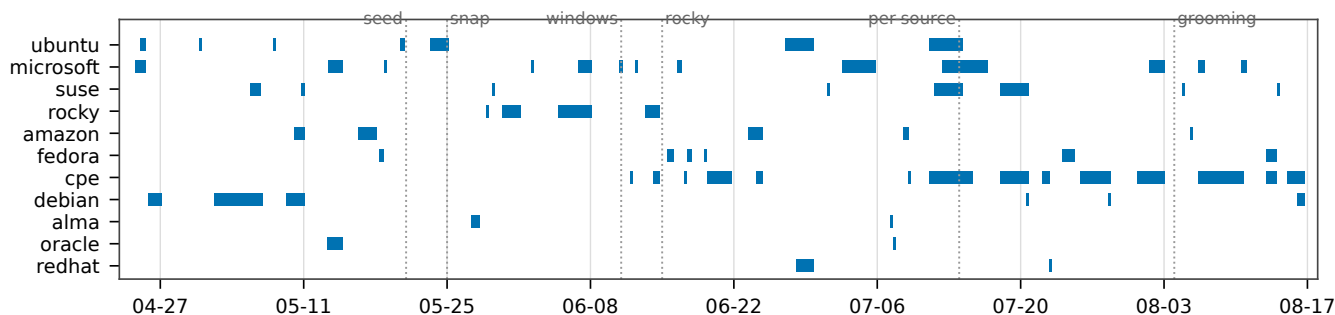


図 3 116 日間の発動タイムライン (source-episode 69 件). 横棒は各 episode の持続期間 (FAIL 初出から終端まで), 点線は各 override(seed, snap, windows, rocky, per-source 化, grooming) の導入時点. per-source 化後の cpe.*系は cpe 行に含めた.

瞬間を取り込めばビルドは成功したまま欠損 DB ができあがる. 公開前の差分検査なしにこれを防ぐ手段は事実上ない.

6.2 遡及的に書き換わる (完全性・一貫性)

- **errata の検知条件の縮退:** AlmaLinux の errata 縮退 (5.3 節) では, フィード, OSV(Open Source Vulnerabilities), HTML の三者が一致したことから配信障害ではないと判断し, 上流の issue tracker へ報告したところ, フィードが現行バージョンの情報しか保持しない仕様に起因するとの説明を得た^{*1}. 上流が仕様どおりに動いていても, 過去バージョンの範囲情報を必要とする脆弱性検知にとってはデータの欠損として働く例である.
- **サイレントなサーバ側再編:** Cisco openVuln API では, アドバイザリの更新日時もバージョンも一切変わらないまま, アドバイザリと影響バージョンのマッピングだけが日次で書き換わり, 93 アドバイザリから製品バージョン 2,022 件が削除された (例: 2014 年のアドバイザリの製品リストが 149 から 36 へ). changelog にも issue にもアナウンスはない. 過剰マッピングの整理というデータ品質改善の側面が強い一方, 該当バージョンを監視している利用者には黙った検知消失となる.
- **一斉再生成の両義性:** VulnCheck の全年代 CVE への生成 CPE 一括付与 (5.3 節) は情報の拡充である一方, CPE 語彙の現代化により既存のマッチングから外れる検知退行を同時に含んでいた. 2 週間後, 上流側の修正により退行の一部は「回復イベント」としてもう一度検査を発動させた. 拡充, 退行, 回復のいずれもが大変動として観測される.

6.3 正当でも桁違いに動く (スケール)

観測された最大変動率は 3433.3%(新規リリース fe-

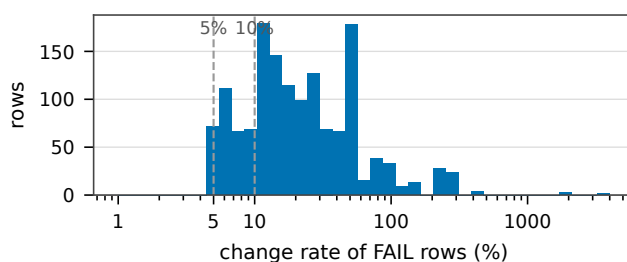


図 4 全 FAIL 行 (1,465 行) の変動率分布 (対数軸). 破線は既定閾値 (detection 5% / db 10%). 閾値近傍の 5~20%帯に対し, 100%超の構造イベント群が裾として分離する. FAIL 行のみを重複 (cron 再実行) 込みで集計した分布であり, 全更新の変動率分布ではない.

dora:46), 428.3%(AlmaLinux errata 再編), 305.9%(CPE 系) に達する一方, 発動に至る日常変動は 5~20%帯に収まり, 両者は概ね桁で分離する. ただし発動記録は閾値で打ち切られた分布であり (図 4), 正常帯全体を含む分離の根拠は 3.1 節のベンチマークにある. ただし分離が崩れる構造要因が 2 つある. (1)baseline が小さい対象は 1 バッチの正当な追加で容易に 100%を超える. (2)FAIL で baseline が停滞すると比較窓が伸び, 日常変動が蓄積して閾値を超える. 前者は per-target 閾値, 後者は迅速な promote 運用で吸収している.

6.4 チェックの相補性と検査の改良

運用からは, チェック間の相補性と, 検査自身の弱点の両方が見えた. 相補性の例が CVRF 消失 (5.2 節の事例 1) である. DB チェックの KB 比較は KB キーの集合しか見ないため, キー集合が不変のまま CVE エントリが大量消失した本事例では 0%を返したが, 観点の異なる Dn が検知結果の激減として捕捉し公開を止めた. 単一の指標には盲点があり, それを補うのが多観点のチェックである (「KB 0% ≠ 変化なし」は運用の警告事項として成文化した). 一方, 弱点として露呈し改良につながったものが 2 つある. (1)ecosystem 一括の変動率は小ソースが大ソースの異常を希釈した (ソース単位への細分化で対処).

^{*1} <https://bugs.almalinux.org/view.php?id=644>

(2)override の系統間非対称が片系統だけの慢性 FAIL を生んだ (grooming で両系統を同時に見ることで対処). 検査は一度作って終わりではなく, 発動事例からのフィードバックで分解能と運用を継続的に改良する対象である. なお (1) のソース単位化の効果は後半の観測窓で実測できた. レポートがソース名を持つため, 前半では「cpe」と一括りだった変動が, vulncheck-nist-nvd2(110 行), cisco-json(42 行), fortinet-csaf(17 行), fortinet-cvrf(10 行), paloalto-json(5 行), jvn-feed-rss(5 行) のように分解され, target 名からソースを推定する必要がなくなった. 5.3 節の VulnCheck 事例と同型の希釈は, 後半の窓では観測されていない.

6.5 運用コストの実態

検査は 943 run 中 420(44.6%) を止め, 手動 promote は 0.79 件/日, ほぼ毎日 1 回人間がレポートを確認して承認している計算になる. run 単位の FAIL の 50%は土日が発生しているが, これは上流ではなく人間側の信号である. promote が平日日中にしか行われなため, 金曜夕方以降のイベントが週末じゅう再 FAIL し続ける (週末滞留). fail sequence の持続時間 (中央値 11h / p90 48h) は検査の検出性能ではなくオペレータの応答時間の分布を測っており, 通知や承認フローの自動化余地を見積もる直接の根拠となる. また保留中は直前版が配られ続けるため, 保留自体が新規 CVE の反映遅延という別種のリスクを生む. 検査の実行時間は DB チェック約 1 分, 検知チェック約 9 分で, 6 時間周期の CI に常設できる.

6.6 開発プロセスに対する最終ガードレールとしての価値

副次的だが実感の大きい価値として, パイプライン開発の最終防衛線がある. テストやレビューが工程ごとのコードの正しさを検査するのに対し, データパイプラインの意味的な正しさは集約後の出力でしか観測できない. 実際, 型の取り違いによるソート処理の欠落 (5.2 節) はコンパイルもテストも通過する類のバグであり, 出力の構造差分としてのみ顕在化した. また, 開発版へ破壊的変更を先行投入して影響規模を定量観測する, 予期した FAIL の発現範囲が想定に閉じていることを混入なしの証拠に使う (5.3 節のピン解除), といった能動的な使い方も定着した. この「作られ方に依存せず生成物を検証する」性質は, AI エージェントがコード生成を担う開発形態で, コードレビューのスケール限界を補う出力レベルのガードレールとして重要性を増すと考える. 本運用でも一次調査は AI エージェントに委ね, 報告の検証と promote の判断は人間に限定している (3.3 節).

7. 関連研究

脆弱性データベースの品質: NVD をはじめとする脆弱性 DB の品質は, 影響バージョン不整合の検出 [5], 品質監

査と自動修正 [6], 国際比較 [7], スキャナや SCA(Software Composition Analysis) ツール間の検知不一致の DB 差への帰着 [8], [9], NVD enrichment 停滞の定量化 [10], [11] など, 外部からの静的測定として繰り返し行われてきた. 本稿は測定器を本番の取り込み・公開パイプラインに常設し, フィードの経時変化を下流消費者の視点で縦断観測した点で異なる.

セキュリティデータフィードの品質測定: 脅威インテリジェンスフィードでは量, カバレッジ, 遅延, 正確性等の比較測定が行われ, フィード間の重複の少なさや品質の不可視性が示されている [12], [13], [14]. いずれも外部からの横断測定であり, 本稿は単一フィードを自身の公開履歴と比較する縦断測定を, 公開を保留する運用機構として常設した点が新しい.

データパイプラインの検証: 宣言的制約によるデータ品質検証 [15], [16] や, パイプラインの実行履歴から検証基準を自動導出する研究 [17], [18] がある. 本稿に最も近いのは, 新しいデータバッチが履歴から逸脱した際に下流の ML 再学習をブロックするゲート [19] で, 「履歴との比較で公開・利用を止める」という意味論まで本稿と同型である. 同ゲートが列統計の分布逸脱を信号として ML の学習品質を保護するのに対し, 本稿の検査は, (1) 検知結果と検知条件構造という生成物の意味レベルの差分を信号とし, (2) セキュリティクリティカルな公開物を監査証跡付きの human-in-the-loop override で保護し, (3) さらに発動記録そのものを上流不安定性の測定データとして全数分析した, という 3 点で異なる. 本稿の検査は前公開版をオラクルとする differential testing[20] の時間軸変種, データへの回帰テスト [21] とみなせる.

国内の関連研究: 国内では, 脆弱性対策情報データベース JVN の提案 [22] 以来の脆弱性情報流通の系譜に, OSS の脆弱性修正状況の調査 [23], PSIRT 向けスクリーニング [24] などが続く. IPA の JVN iPedia 登録状況レポート [25] は 2024 年の NVD 公開遅延が国内 DB の登録数を減少させたことを記録しており, 上流の不安定性が国内基盤へ波及する事例である. フィード内容の経時的不安定性を定量計測した国内の学術研究は, 我々の調査の範囲では見当たらない.

8. おわりに

脆弱性 DB の公開 CI に公開前検査を実装し, 116 日間・943 run の全数調査から 69 件の発動 episode を抽出して, 59 件を上流由来, 10 件を自前の処理由由来に帰属した. 上流は消え, 戻り, 遡及的に書き換わり, アナウンスなく再編される. しかもその大半は正当な変更である. 得られた知見は次の通りである. (1) 正常変動と異常イベントは変動率で概ね桁分離し, 単純な閾値検査が実用になる. ただし分解能 (ソース単位) と閾値 (対象単位の override と定期

grooming) は運用実績から継続的に導出する必要がある。(2) 発動原因の帰属は、収集・変換・構築の全工程が履歴管理されていることを成立条件とする。前稿で報告した履歴管理基盤 [2] は、本測定と公開品質管理の土台として実証された。本稿は同稿が課題として挙げた「差分の論理的な提示」「パイプラインの堅牢化」への、運用を伴う一つの回答でもある。(3) FAIL 時の復旧は「人間によるレポート確認+監査証跡付き override」として設計すべきである。誰がどの候補を通したかが履歴に残ることが、安全ネットを緩める操作の正当性を支える。(4) 上流の恒久的劣化には、取り込みのピン留めや自らの取得履歴からの復元という出口が要る。(5) 生成物を出力レベルで検証する検査は、自らの開発プロセスに対する最終防衛線としても機能し、テストとレビューをすり抜ける意味的バグの捕捉に実際に寄与した。コード生成に AI が入り込む開発形態でこの価値は増すと考える。

今後の課題として、fetch 段階での欠損検出 (月次ドキュメント丸ごと消失時にコミットしない等)、通知や承認フローの改善による週末滞留の解消、観測の長期蓄積による時系列特性の統計的確立、変動率の経過時間正規化による停滞バイアスの緩和、および同一上流を消費する他エコシステムとの観測の突合が挙げられる。

参考文献

- [1] Vulns: Vulnerability scanner. <https://github.com/future-architect/vulns>.
- [2] 中岡典弘, 篠原俊一. 広範な脆弱性情報の統合管理と履歴追跡. コンピュータセキュリティシンポジウム 2025 (CSS 2025), 2025.
- [3] vuls-data-update. <https://github.com/MaineK00n/vuls-data-update>.
- [4] vuls-data-db. <https://github.com/vulsio/vuls-data-db>.
- [5] Ying Dong, Wenbo Guo, Yueqi Chen, Xinyu Xing, Yuqing Zhang, and Gang Wang. Towards the detection of inconsistencies in public security vulnerability reports. In *28th USENIX Security Symposium*, pp. 869–885, 2019.
- [6] Afsah Anwar, Ahmed Abusnaina, Songqing Chen, Frank Li, and David Mohaisen. Cleaning the NVD: Comprehensive quality assessment, improvements, and analyses. *IEEE Transactions on Dependable and Secure Computing*, Vol. 19, No. 6, pp. 4255–4269, 2022.
- [7] Juehao Lin, Xuanxiang William Wang, Gianluca Stringhini, and Manuel Egele. A comparative analysis of nvd, jvndb and cnvd: Insights into global and regional vulnerability reporting. In *Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2026.
- [8] Nasif Imtiaz, Seaver Thorn, and Laurie Williams. A comparative study of vulnerability reporting by software composition analysis tools. In *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2021.
- [9] Yekaterina Churakova, Mathias Ekstedt, and Larissa Schmid. Vexed by VEX tools: Consistency evaluation of container vulnerability scanners. arXiv:2503.14388, 2025.
- [10] VulnCheck. The real danger lurking in the NVD backlog. <https://www.vulncheck.com/blog/nvd-backlog-exploitation>, 2024. (Accessed 2026-07-29).
- [11] Anchore. The NVD enrichment crisis: One year later. <https://anchore.com/blog/nvd-crisis-one-year-later/>, 2025. (Accessed 2026-07-29).
- [12] Vector Guo Li, Matthew Dunn, Paul Pearce, Damon McCoy, Geoffrey M. Voelker, Stefan Savage, and Kirill Levchenko. Reading the tea leaves: A comparative analysis of threat intelligence. In *28th USENIX Security Symposium*, pp. 851–867, 2019.
- [13] Xander Bouwman, Harm Griffioen, Jelle Egbers, Christian Doerr, Bram Klievink, and Michel van Eeten. A different cup of TI? the added value of commercial threat intelligence. In *29th USENIX Security Symposium*, 2020.
- [14] Harm Griffioen, Tim Booij, and Christian Doerr. Quality evaluation of cyber threat intelligence feeds. In *Applied Cryptography and Network Security (ACNS), LNCS 12147*, pp. 277–296, 2020.
- [15] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. Automating large-scale data quality verification. *Proceedings of the VLDB Endowment*, Vol. 11, No. 12, pp. 1781–1794, 2018.
- [16] Eric Breck, Marty Zinkevich, Neoklis Polyzotis, Steven Euijong Whang, and Sudip Roy. Data validation for machine learning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2019.
- [17] Dezhan Tu, Yeye He, Weiwei Cui, Song Ge, Haidong Zhang, Shi Han, Dongmei Zhang, and Surajit Chaudhuri. Auto-validate by-history: Auto-program data quality constraints to validate recurring data pipelines. In *29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2023.
- [18] Sergey Redyuk, Zoi Kaoudi, Volker Markl, and Sebastian Schelter. Automating data quality validation for dynamic data ingestion. In *24th International Conference on Extending Database Technology (EDBT)*, pp. 61–72, 2021.
- [19] Shreya Shankar, Labib Fawaz, Karl Gyllstrom, and Aditya G. Parameswaran. Moving fast with broken data. arXiv:2303.06094, 2023.
- [20] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, Vol. 10, No. 1, pp. 100–107, 1998.
- [21] Shin Yoo and Mark Harman. Regression testing minimization, selection and prioritization: A survey. *Software Testing, Verification and Reliability*, Vol. 22, No. 2, pp. 67–120, 2012.
- [22] 寺田真敏, 高田真吾, 土居範久. 脆弱性対策情報データベース JVN の提案. 情報処理学会論文誌, Vol. 46, No. 5, pp. 1256–1265, 2005.
- [23] 葛野弘樹, 矢野智彦, 面和毅, 山内利宏. オープンソースソフトウェアを対象とした脆弱性修正状況の収集と調査. コンピュータセキュリティシンポジウム 2024 (CSS 2024), 2024.
- [24] 金井遵, 上原龍也, 小池竜一, 鬼頭利之, 神戸康多, 木戸俊輔, 篠原俊一, 中岡典弘, 林優二郎. PSIRT 向け脆弱性スクリーニング技術と環境毎リスク評価技術. コンピュータセキュリティシンポジウム 2024 (CSS 2024), 2024.
- [25] 情報処理推進機構. 脆弱性対策情報データベース JVN iPedia の登録状況. <https://www.ipa.go.jp/security/reports/vuln/jvn/>. 四半期レポート.